

Uses and Properties

The Precision and Framework of Programming Languages



Diversity

Numerous programming languages in the computing realm



Precision

Higher level of precision than natural languages



Framework

Systematic framework for specifying information and operations

Diversity of Programming Languages

Numerous different programming languages have been developed, mostly in the realm of computing.

Language Proliferation

Individual software projects frequently employ five or more programming languages, each serving different purposes:



Frontend

User interface and
experience



Backend

Server-side logic and
databases



System

Low-level hardware
interaction

```
// Example of C in a multi-language project
#include <stdio.h>
#include <stdlib.h>

// C might be used for performance-critical components
void process_data(int *data, int size) {
    // High-performance data processing
    for (int i = 0; i < size; i++) {
        data[i] = data[i] * 2 + 1;
    }
}

// This C function could be called from Python, Java, or other languages
// using foreign function interfaces (FFI)
```



Real-World Example

A modern web application might use HTML/CSS/JavaScript for the frontend, Python for backend services, SQL for database queries, and C for performance-critical components like image processing or cryptographic operations.

2/6

Precision and Completeness

Programming languages, unlike most other forms of human expression, need a higher level of precision and completeness.

Computers Do Exactly What They're Told

When communicating with others in a natural language, human writers and speakers might be unclear and make minor mistakes while still expecting their purpose to be understood. Computers, on the other hand, "do exactly what they are told to do" and cannot "understand" the code the programmer intended to create.

The language definition, a program, and the program's inputs must properly define the external behavior that occurs when the program is performed, inside the program's domain of control.

```
// Precision in C - small mistakes have big consequences
int calculate_average(int a, int b) {
    // Correct implementation
    return (a + b) / 2;
}

int calculate_average_wrong(int a, int b) {
```

```
// Common mistake - integer division before addition
return a + b / 2;
}

// calculate_average(5, 7) returns 6 (correct)
// calculate_average_wrong(5, 7) returns 8 (incorrect)
```



Pseudocode Communication

By employing pseudocode, which interleaves plain language with code written in a programming language, concepts about an algorithm may be communicated to users without the accuracy necessary for execution.

3/6

Framework for Information and Operations

The programming language gives a systematic framework for specifying information as well as the operations or modifications that may be performed on that data automatically.

Representing Concepts

A programmer represents the notions involved in a calculation using the abstractions provided by the language. These ideas are represented by a collection of the most basic materials accessible.

```
// C provides a framework for representing information and operations
#include <stdio.h>
#include <string.h>
```

```
// Information representation through data structures
typedef struct {
    char name[50];
    int age;
    float gpa;
} Student;

// Operations on the information
void print_student(Student s) {
    printf("Name: %s, Age: %d, GPA: %.2f\n", s.name, s.age, s.gpa);
}

void update_gpa(Student *s, float new_gpa) {
    s->gpa = new_gpa;
}

int main() {
    Student alice = {"Alice", 20, 3.7};
    print_student(alice);
    update_gpa(&alice, 3.9);
    print_student(alice);
    return 0;
}
```



Real-World Example

In C, the struct keyword provides a framework for combining related data into a single unit, while functions provide a framework for defining operations that can be performed on that data. This systematic approach allows programmers to model real-world concepts in code.

4/6

Programming as a Process

The procedure by which programmers combine these primitives to create new programs or modify current ones to new applications or a changing environment is known as programming.

The Programming Process

1 Understand the Problem – Analyze requirements and constraints

2 Design the Solution – Plan the structure and approach

3 Implement the Code – Write the actual program using language primitives

4 Test and Debug – Verify correctness and fix issues

5 Maintain and Evolve – Adapt to changing requirements

```
// Example of evolving a program to meet new requirements

// Initial version - simple calculator
int add(int a, int b) {
    return a + b;
}

// Evolved version - calculator with error handling
int add_safe(int a, int b, int *result) {
    // Check for integer overflow
    if ((b > 0 && a > INT_MAX - b) || (b < 0 && a < INT_MIN - b)) {
        return 0; // Error: overflow would occur
    }
    *result = a + b;
    return 1; // Success
}

// Further evolution - supporting multiple operations
typedef enum { ADD, SUBTRACT, MULTIPLY, DIVIDE } Operation;

int calculate(int a, int b, Operation op, int *result) {
    switch (op) {
        case ADD: return add_safe(a, b, result);
        case SUBTRACT: // Implementation for subtraction
        case MULTIPLY: // Implementation for multiplication
        case DIVIDE: // Implementation for division
        default: return 0; // Error: unknown operation
    }
}
```

Execution Modes and Scripting Languages ⚡

Computer programs can be run in different modes, with scripting languages offering a unique approach to execution.

Execution Modes

Batch Process



Programs can be run without human intervention, executing a predefined sequence of operations.

Interactive Session



A user can input commands in an interactive session with an interpreter. The "commands" in this scenario are merely programs whose execution is linked together.

```
// Example of C in both batch and interactive modes

// Batch mode example - a program that processes a file
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    if (argc != 2) {
        printf("Usage: %s \n", argv[0]);
        return 1;
    }

    FILE *file = fopen(argv[1], "r");
    if (!file) {
        printf("Error opening file\n");
        return 1;
    }
}
```



```

}

// Process file contents...
fclose(file);
return 0;
}

// Interactive mode example - a simple calculator
#include <stdio.h>

int main() {
    int a, b;
    char op;

    printf("Simple Calculator\n");
    printf("Enter expression (e.g., 5 + 3): ");
    scanf("%d %c %d", &a, &op, &b);

    switch (op) {
        case '+': printf("Result: %d\n", a + b); break;
        case '-': printf("Result: %d\n", a - b); break;
        case '*': printf("Result: %d\n", a * b); break;
        case '/': printf("Result: %.2f\n", (float)a / b); break;
        default: printf("Unknown operator\n");
    }

    return 0;
}

```

Scripting Languages

A scripting language is one that can run its instructions through an interpreter without compiling. While C is typically a compiled language, it can be used in scripting contexts through tools like Cint or C-REPL, and it often serves as the foundation for implementing interpreters of scripting languages.